TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

PURWANCHAL CAMPUS

A FINAL MAJOR PROJECT REPORT ON BUILDING A LEGAL CHATBOT USING RAG AND QLORA FINE-TUNING

BY

Nishan Bhattarai (PUR078BCT054)

Prasanga Niraula (PUR078BCT056)

Rajat Pradhan (PUR078BCT059)

Yamraj Khadka (PUR078BCT096)

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

PURWANCHAL CAMPUS

DHARAN, NEPAL

MARCH, 2026

COPYRIGHT©

The author has agreed that the Library, Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purpose may be granted by the supervisor(s) who supervised the thesis work recorded herein or, in their absence, by the Head of the Department wherein the thesis report was done. It is understood that the recognition will be given to the author of this report and to the Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering in any use of the material of this thesis report. Copying or publication or the other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering and author's written permission is prohibited. Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head
Department of Electronics and Computer Engineering
Purwanchal Campus, Institute of Engineering
Dharan , Sunsari
Nepal

DECLARATION

We declare that the work hereby submitted for Bachelors of Engineering in Computer Engineering at Institute of Engineering, Purwanchal Campus with the title **"LEGAL CHATBOT FOR NEPALESE LAW USING FINE-TUNED LLM AND RAG"** is our own work and has not been previously submitted by me at any university for any academic award.

We authorize Institute of Engineering, Purwanchal Campus to lend this report to other institution or individuals for the purpose of academic research.

STUDENTS NAME:
Nishan Bhattarai (PUR078/BCT/054)
Prasanga Niraula (PUR078/BCT/056)
Rajat Pradhan (PUR078/BCT/059)
Yamraj Khadka (PUR078/BCT/096)

ACKNOWLEDGEMENT

First and foremost, we would like to express our heartfelt gratitude and appreciation to all those who have supported us throughout the development of this project.

We express our sincere admiration to the **Head of the Department(HOD) Er. Manoj Kumar Guragain** and our **Project Supervisor Er. Binay Lal Shrestha** for their valuable unwavering support, diligent guidance, constructive criticisms and resourceful feedback.

We are deeply thankful to the esteemed teachers and faculty members whose valuable feedback, encouragement, and meaningful insights have enriched this proposal. Their dedication to academic excellence and commitment to fostering knowledge have been a source of inspiration. Their collective guidance have immensely contributed to the refinement and improvement of this proposal.

STUDENTS NAME:

Nishan Bhattarai (PUR078/BCT/054)

Prasanga Niraula (PUR078/BCT/056)

Rajat Pradhan (PUR078/BCT/059)

Yamraj Khadka (PUR078/BCT/096)

ABSTRACT

This project presents the creation of a domain-specific legal language model that can provide context-aware answers to questions about Nepalese law. Large language models from important legal texts, were used to curate the dataset. Using the parameter-efficient QLoRA method, a LLM was fine-tuned using the curated dataset. A simple Retrieval-Augmented Generation (RAG) pipeline that combines vector similarity search, dense retrieval, keyword boosting, and re-ranking mechanisms was put into place to improve factual accuracy and grounding in legal context. The final system shows that it can generate logical, legally sound answers while maintaining computational efficiency.

Keywords: LLMs, QLoRA, RAG, Nepalese Law, Legal NLP

TABLE OF CONTENTS

COPYRIGHT© i

DECLARATION ii

ACKNOWLEDGEMENT iii

ABSTRACT iv

LIST OF FIGURES xi

LIST OF TABLES xii

LIST OF ABBREVIATIONS xiii

1 INTRODUCTION 1

1.1 Background 1

1.2 Gap Identification 1

1.3 Motivation 2

1.4 Objectives 2

2 RELATED THEORY 3

2.1 Large Language Models and Transfer Learning 3

2.1.1 How does a Large Language Model work? 3

2.2 Fine-Tuning Large Language Models (LLMs) 7

2.2.1 Types of Fine-Tuning 8

2.3 Retrieval-Augmented Generation (RAG) 11

2.3.1 Overall Pipeline Architecture 12

2.3.2 Step 1: Data Collection and Preprocessing 12

2.3.3 Step 2: Data Chunking and Representation 13

2.3.4 Step 3: Embedding Generation 14

2.3.5 Step 4: Dense Retrieval 16

2.3.6 Step 5: Reranking 16

2.3.7 Step 6: Hybrid Retrieval 17

2.3.8 Step 7: Context Integration and Response Generation 18

2.4 Instruction Tuning and Prompt-Based Learning 18

2.5 Model Quantization 19

2.5.1 Quantization Overview 19

2.5.2 4-bit Quantization 20

2.5.3 GGUF Quantization Format 20

2.5.4 Benefits and Trade-offs 20

3 LITERATURE REVIEW **22**

3.1 Legal LLM Fine-Tuning Using LoRA and QLoRA in South Asia 22

3.2 Research Gap in the Nepali Legal Context 23

4 METHODOLOGY **25**

4.1 Overview 25

4.2 Stage 1: Data Preparation and Structuring 26

4.2.1 Hierarchical Extraction 26

4.3 Stage 2: Instruction Dataset Generation 27

4.4 Stage 3: Model Fine-Tuning using QLoRA 27

4.4.1 LoRA Configuration 28

4.4.2 Training Arguments 30

4.5 Stage 4: Model Optimization (Adapter Merging and Quantization) 32

4.5.1 LoRA Adapter Merging 32

4.5.2 GGUF Conversion and 4-bit Quantization 32

4.6 Stage 5: Retrieval-Augmented Generation (RAG) Pipeline 33

4.7 Stage 6: System Architecture (Frontend and Backend) 34

4.7.1 System Overview 34

4.7.2 Backend Architecture 34

4.7.3 Frontend Architecture 34

4.7.4 Frontend-Backend Interaction Flow 35

4.7.5 System Flow Diagram 36

4.7.6 Advantages of Separation 36

4.8 Stage 7: Evaluation Strategy 37

4.8.1 Quantitative Metrics 37

4.8.2 Custom Benchmarking 38

4.9 Stage 8: Deployment 38

5 EVALUATION AND BENCHMARKING 39

5.1 Model Training Overview 39

5.1.1 Training and Validation Loss 39

5.1.2 Perplexity Analysis 40

5.2 Limitations of Standard Metrics 41

5.3 Custom Benchmarking Parameters 42

5.4 Evaluation Framework and Scoring Metrics 42

5.5 Models Under Test 43

5.6 Benchmarking Run 1: Without RAG 44

5.6.1 Overall Scores (No RAG) 44

5.6.2 Per Category Breakdown (No RAG) 45

5.6.3 Radar Chart for No RAG 46

5.6.4 Notable Observations (Without RAG) 46

5.7 Benchmarking Run 2: With RAG 48

5.7.1 Overall Scores (With RAG) 48

5.7.2 Per Category Breakdown (With RAG) 49

5.7.3 Radar Chart for RAG based implementation 50

5.7.4 Noteable Observation (With RAG) 50

5.8 RAG Impact Assessment 52

5.8.1 Score Improvements from RAG 52

5.8.2 Per Category Score Improvements from RAG 52

5.8.3 Notable Observations from RAG Score Improvements 53

5.9 Final Verdict 53

REFERENCES **55**

LIST OF FIGURES

Figure 2.1: LLM architecture overview 8

Figure 2.2: Working of RAG 11

Figure 2.3: Embedding Model 15

Figure 4.1: Frontend-Backend Interaction in the Web-Based Legal Assistant System 36

Figure 5.1: Training vs Validation Loss over training steps 40

Figure 5.2: Perplexity over training steps 41

Figure 5.3: Evaluation Framework 42

Figure 5.4: Models Under Testing 43

Figure 5.5: No RAG Benchmarking 44

Figure 5.6: No RAG Benchmarking with categories 45

Figure 5.7: Radar Chart (No RAG) 46

Figure 5.8: Benchmarking Score with RAG 48

Figure 5.9: Benchmarking with categories using RAG 49

Figure 5.10:Radar Chart (With RAG) 50

Figure 5.11:Score Improvement with RAG 52

Figure 5.12:Per Category Score Improvement with RAG 52

LIST OF TABLES

LIST OF ABBREVIATIONS

AI	: Artificial Intelligence
BERT	: Bidirectional Encoder Representations from Transformers
CA	: Contextual Accuracy
CMA	: Clause Mapping Accuracy
GPT	: Generative Pre-trained Transformer
GGUF	: GPT-Generated Unified Format
LLM	: Large Language Model
LLaMA	: Large Language Model Meta AI
LoRA	: Low-Rank Adaptation
NLP	: Natural Language Processing
PEFT	: Parameter-Efficient Fine-Tuning
QLoRA	: Quantized Low-Rank Adaptation
RAG	: Retrieval Augmented Generation
ROGUE	: Recall-Oriented Understudy for Gisting Evaluation
WMS	: Weighted Model Score

CHAPTER 1

INTRODUCTION

1.1 Background

In recent years, the development of LLMs has revolutionized how information is processed, retrieved, and understood. These models, when fine-tuned on domain-specific data, have demonstrated remarkable capability in natural language understanding, reasoning, and generation tasks. Meanwhile, access to legal knowledge — especially in the context of Nepal's legal framework — remains complex, primarily due to the textual density of legal documents and the lack of AI-powered tools tailored to Nepali jurisprudence.

Nepal's primary legal statutes, such as the Muluki Dewani Samhita (Civil Code) and Muluki Aparadh Samhita (Criminal Code), are foundational documents governing a wide range of legal procedures and rights. While digital versions exist in English and Nepali, they are not easily accessible in an interactive or intelligent format that supports summarization, question-answering, or legal reasoning.

1.2 Gap Identification

Despite the growth of AI in legal tech globally, Nepal lacks:

- A domain-specific legal language model fine-tuned on its national legal codes.
- An intelligent assistant capable of understanding and answering user queries based on the Muluki Samhita.
- Tools that provide legal interpretability and support for the general public, students,

or practitioners using AI.

- No current solution leverages QLoRA-based fine-tuning or instruction tuning techniques to make these documents interactively usable via a conversational AI assistant.

1.3 Motivation

The motivation behind this project stems from:

- The need to democratize access to legal knowledge through intelligent systems.
- The opportunity to use QLoRA to adapt existing open-source LLMs for Nepali legal applications.
- A desire to bridge the gap between legal literacy and AI, especially in low-resource language environments like Nepali law.
- Advancing academic research and practical deployment of AI-powered legal assistants in South Asian legal contexts.

1.4 Objectives

- To fine-tune a quantized large language model of about 8B parameters using QLoRA and instruction tuning techniques on the Muluki Civil and Criminal Codes.
- To deploy the fine-tuned model as an AI-powered legal assistant capable of interacting with users for legal queries and summaries.

CHAPTER 2

RELATED THEORY

This chapter reviews the theoretical foundations relevant to this project. We discuss large language models (LLMs), parameter-efficient fine-tuning techniques, retrieval-augmented generation, and instruction tuning. These concepts provide the basis for developing a Nepalese legal assistant using QLoRA and hybrid retrieval.

2.1 Large Language Models and Transfer Learning

Large Language Models (LLMs) such as GPT, BERT, LLaMA, and Mistral are based on the transformer architecture, which introduced the self-attention mechanism for modeling long-range dependencies in text. These models are trained on massive corpora to learn general language patterns, reasoning capabilities, and contextual understanding.

Transfer learning allows these models to be adapted to new, domain-specific tasks — such as legal summarization or question-answering — with significantly less data than was used in pretraining. By fine-tuning LLMs on legal texts, we can transfer the general language understanding of the base model into a highly specialized model that understands the structure and language of Nepal's legal system. This approach is especially effective in low-resource languages or niche domains where large-scale datasets are scarce.

2.1.1 How does a Large Language Model work?

A Large Language Model (LLM) generates and understands text by modeling the probability distribution of token sequences. Given a sequence of tokens, the model predicts

the most likely next token based on the context. This process is enabled by the transformer architecture, which uses self-attention to capture relationships between all tokens in a sequence.

The working of an LLM can be understood as the following step-by-step pipeline:

1. **Tokenization**

   The input text is first converted into smaller units called **tokens**. These may be words, subwords, or characters depending on the tokenizer.

   Example: "punishment for theft" $\rightarrow$ ["punish", "ment", "for", "theft"]

   Tokenization helps handle rare words and reduces vocabulary size while preserving meaning.

2. **Embedding Layer**

   Each token is mapped to a dense vector representation in a high-dimensional space:

$$\mathbf{x}_i = \text{Embed}(t_i), \quad \mathbf{x}_i \in \mathbb{R}^d \tag{2.1}$$

   Example: Consider the tokens "theft" and "stealing". After embedding:

   Embed("theft") $\approx$ [0.21, -0.34, 0.78, ...]

   Embed("stealing") $\approx$ [0.19, -0.30, 0.81, ...]

   These embeddings capture semantic relationships between tokens. For instance, tokens with similar meanings tend to have similar vector representations.

3. **Positional Encoding**

   Since the transformer processes tokens in parallel and does not inherently understand order, positional encodings are added to the embeddings:

$$\mathbf{z}_i = \mathbf{x}_i + \mathbf{p}_i \tag{2.2}$$

where $\mathbf{p}_i$ encodes the position of the token in the sequence. This allows the model to distinguish between different word orders.

4. **Self-Attention Mechanism**

   The core component of an LLM is the self-attention mechanism, which allows each token to attend to all other tokens in the sequence and capture contextual relationships.

   For an input matrix X, three projections are computed:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \tag{2.3}$$

   Attention scores are calculated as:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.4}$$

   *E*ach token determines how much attention to pay to other tokens. For example, in "The judge gave the verdict because *he* reviewed the case", the model learns that "he" refers to "judge".

5. **Multi-Head Attention**

   Instead of a single attention operation, the model uses multiple attention heads:

$$\text{MultiHead}(Q,K,V) = \text{Concat}(head_1, \ldots, head_h)W_O \tag{2.5}$$

Each head captures different types of relationships (e.g., syntactic structure, semantic similarity), improving the model's representational power.

6. **Feed-Forward Networks and Residual Connections**

   Each transformer layer includes a position-wise feed-forward network:

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2.6}$$

   Residual connections and layer normalization are applied to stabilize training and improve gradient flow.

   Intuition: These layers refine token representations and allow the model to learn complex patterns beyond attention.

7. **Stacking Transformer Layers**

   Multiple transformer layers are stacked to form a deep network. Each layer progressively refines the representation of the input sequence, enabling the model to capture higher-level abstractions such as grammar, reasoning, and domain-specific knowledge.

8. **Next Token Prediction**

   The final hidden representation is passed through a linear layer and softmax function to produce a probability distribution over the vocabulary:

$$P(t_{i+1} \mid t_1, t_2, \ldots, t_i) = \mathrm{softmax}(h_i W_o) \tag{2.7}$$

   The model selects the most probable next token (or samples from the distribution).

9. **Autoregressive Text Generation**

The predicted token is appended to the sequence, and the process repeats iteratively:

$$(t_1, t_2, \ldots, t_i) \rightarrow t_{i+1} \rightarrow t_{i+2} \rightarrow \ldots \tag{2.8}$$

This autoregressive process continues until a stopping condition is met (e.g., end-of-sequence token or maximum length).

Overall Intuition: An LLM reads the entire input, understands relationships between tokens using attention, and generates text one token at a time based on learned probability distributions.

Summary: Large Language Models operate by transforming text into embeddings, processing them through multiple self-attention layers, and iteratively predicting the next token. This enables coherent and context-aware generation.

2.2 Fine-Tuning Large Language Models (LLMs)

Large Language Models (LLMs) are pretrained on massive general-purpose corpora. While they capture general language understanding, domain-specific tasks—such as legal question answering, clause summarization, or legal reasoning—require adaptation. Fine-tuning allows the model to specialize in a target domain by updating its parameters based on domain-specific data.

Fine-tuning enables the model to:

- Learn the structure and language of domain-specific texts
- Improve accuracy and relevance of outputs
- Align generation with task-specific requirements, such as summarization or instruction-

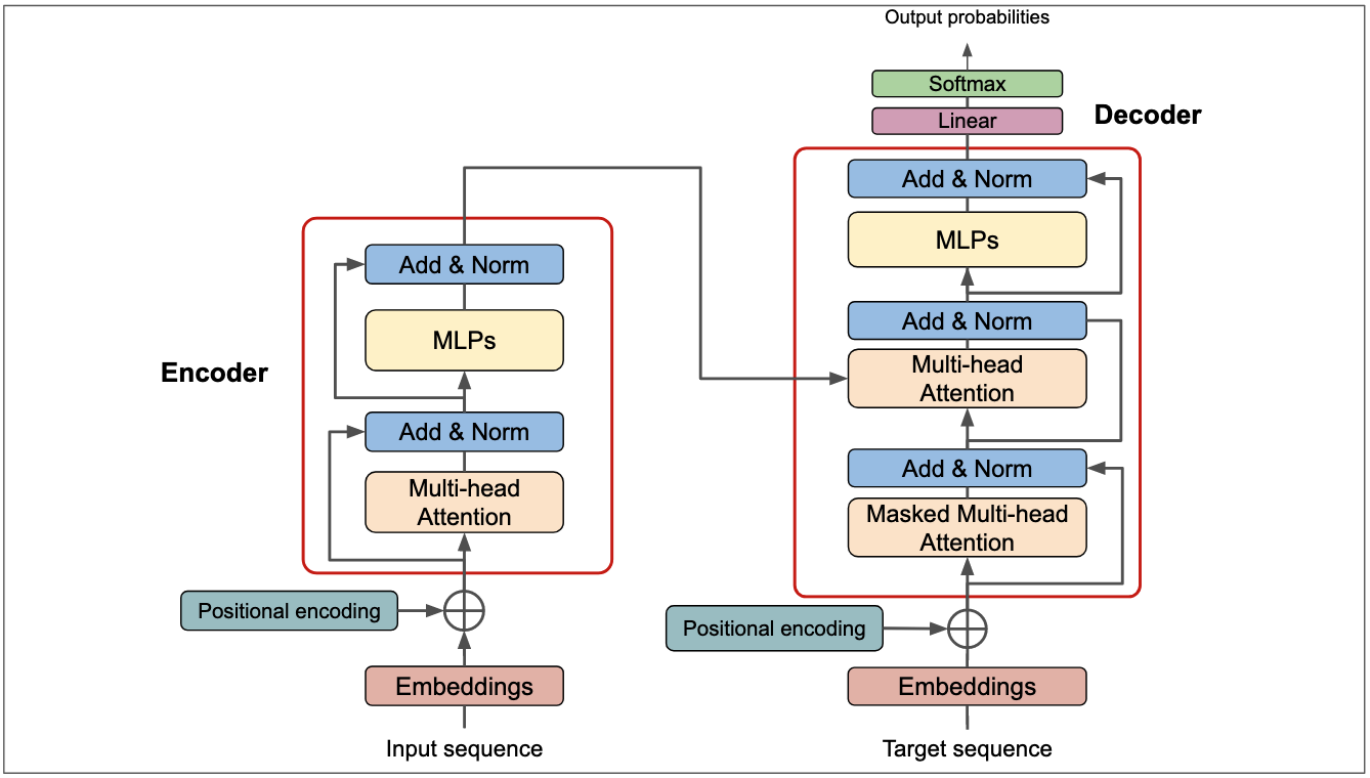

Figure 2.1: LLM architecture overview

following

2.2.1 Types of Fine-Tuning

1. **Full Fine-Tuning**

   Full fine-tuning of large language models involves updating all model parameters during training, which results in significant computational cost and high memory requirements. This process becomes increasingly impractical as model sizes scale to billions of parameters, requiring specialized hardware and extended training time.

   Such approaches demand substantial GPU memory for storing model weights, gra-

dients, and optimizer states, making them resource-intensive and less accessible for many real-world applications. Additionally, full fine-tuning can lead to inefficiencies when adapting models to specific domains, as the entire parameter space is modified even when only a subset of knowledge needs adjustment.

Therefore, while full fine-tuning allows comprehensive adaptation of the model, it presents challenges in terms of scalability, efficiency, and deployment feasibility, particularly in constrained environments.

Mathematically, if W represents the model's weights and $\mathscr{L}$ the loss on the target dataset:

$$W_{\text{new}} = \arg\min_{W} \mathscr{L}(W; \text{dataset}) \tag{2.9}$$

2. **Parameter-Efficient Fine-Tuning (LoRA / QLoRA)**

   Parameter-efficient fine-tuning (PEFT) focuses on adapting large language models by updating only a small subset of parameters while keeping the majority of pretrained weights frozen. Instead of modifying the entire model, PEFT introduces lightweight, trainable components that enable the model to learn task-specific patterns with minimal computational overhead.

   By restricting updates to a limited set of parameters, PEFT significantly reduces memory consumption, training time, and hardware requirements. This makes it particularly suitable for large-scale models, where full parameter updates would otherwise be prohibitively expensive.

   Moreover, freezing the core pretrained weights helps preserve the general knowledge learned during pretraining, while the additional trainable parameters specialize

the model for a target domain. This results in efficient adaptation without degrading the model's original capabilities.

- **LoRA (Low-Rank Adaptation):** LoRA freezes the original model weights and injects trainable low-rank adapters in key layers, typically in the attention modules. Mathematically:

$$W_{\text{eff}} = W_{\text{base}} + \Delta W_{\text{LoRA}} = W_{\text{base}} + AB \tag{2.10}$$

  where:

  - $W_{\text{base}} \in \mathbb{R}^{d \times k}$ is the frozen weight matrix,
  - $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$ are trainable low-rank matrices ($r \ll \min(d,k)$),
  - $\Delta W_{\text{LoRA}} = AB$ is the low-rank update added to W_{base}.

  This allows the model to adapt to new tasks without updating the entire weight matrix.

- **QLoRA:** QLoRA extends LoRA by quantizing base weights to 4-bit precision while keeping LoRA adapters in full precision:

$$W_{\text{eff}} = W_{\text{4bit}} + \Delta W_{\text{LoRA}} \tag{2.11}$$

  This approach drastically reduces memory usage and enables training on modest hardware without compromising performance.

2.3 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a hybrid framework that enhances the capability of Large Language Models (LLMs) by integrating external knowledge retrieval into the text generation process. Traditional LLMs rely solely on knowledge encoded within their parameters during pretraining, which often leads to issues such as hallucination, outdated information, and lack of domain specificity. RAG addresses these limitations by dynamically retrieving relevant information from a structured knowledge base and conditioning the generation process on this retrieved context.

This paradigm is particularly critical in the legal domain, where correctness, traceability, and adherence to statutory text are essential. By grounding model outputs in retrieved legal clauses (e.g., from the Muluki Civil Code or Criminal Code), RAG ensures that generated responses remain faithful to authoritative sources.

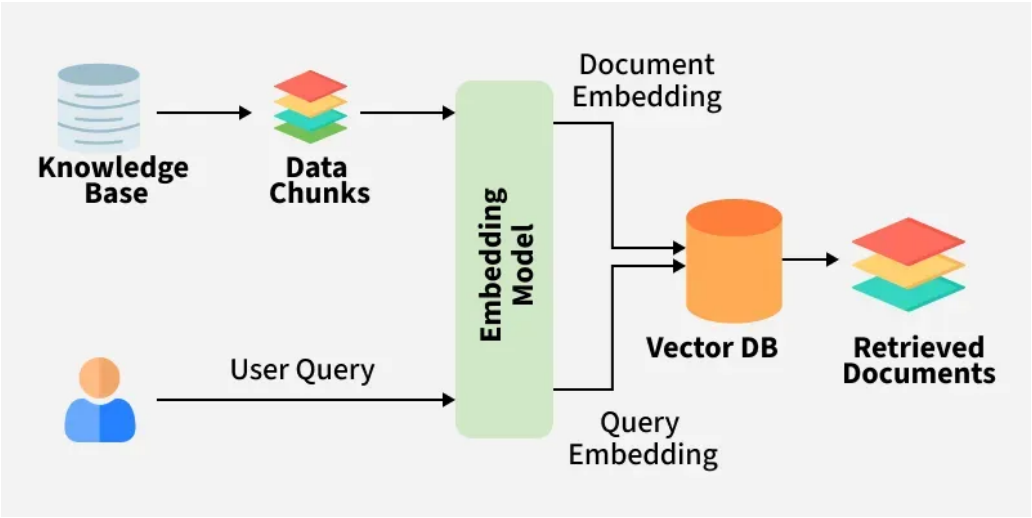

Figure 2.2: Working of RAG

The RAG pipeline can be viewed as a sequence of interconnected stages, each responsible

for transforming raw textual data into meaningful, context-aware responses.

2.3.1 Overall Pipeline Architecture

The proposed RAG system follows a structured multi-stage pipeline:

1. Data Collection and Preprocessing
2. Data Chunking and Structuring
3. Embedding Generation
4. Query Processing and Dense Retrieval
5. Reranking and Hybrid Retrieval
6. Context Integration and Response Generation

Each stage plays a crucial role in ensuring that the system retrieves the most relevant legal information and generates accurate responses.

2.3.2 Step 1: Data Collection and Preprocessing

The foundation of any RAG system is a high-quality domain-specific corpus. In this work, the dataset consists of structured legal documents, including statutory provisions, clauses, and legal definitions extracted from Nepali legal codes.

The preprocessing stage involves multiple transformations to standardize and clean the data:

- **Text Normalization:** Removal of noise such as inconsistent formatting, special characters, and redundant whitespace.
- **Structural Parsing:** Identification and extraction of clause boundaries, section

headings, and clause identifiers.

- **Metadata Enrichment:** Associating each clause with relevant attributes such as clause ID, category (civil/criminal), and hierarchical relationships.

This step ensures that the knowledge base is both machine-readable and semantically structured, enabling efficient retrieval.

2.3.3 Step 2: Data Chunking and Representation

Legal documents are inherently long and complex, often containing multiple clauses within a single section. Directly embedding entire documents can dilute semantic meaning and reduce retrieval precision. Therefore, documents are segmented into smaller, semantically coherent units.

Chunking is performed based on:

- Clause-level segmentation
- Logical sentence grouping
- Context-preserving windowing (optional overlapping chunks)

Let the corpus be represented as:

$$\mathscr{D} = \{d_1, d_2, \ldots, d_n\} \tag{2.12}$$

where each d_i represents a chunked legal clause.

Chunking improves:

- Retrieval granularity

- Semantic precision
- Computational efficiency

2.3.4 Step 3: Embedding Generation

To enable semantic search, textual data must be transformed into numerical vector representations known as embeddings. These embeddings encode the meaning of text in a high-dimensional vector space.

Embedding Function

Each document chunk and query is mapped to a dense vector:

$$\mathbf{e}_i = \text{Embed}(d_i), \quad \mathbf{e}_q = \text{Embed}(q) \tag{2.13}$$

where $\text{Embed}(\cdot)$ is a neural encoder (e.g., Sentence-BERT or a fine-tuned embedding model).

Types of Embeddings

Different embedding strategies capture varying levels of semantic information:

- **Static Word Embeddings:** Capture general semantic relationships but lack contextual awareness.
- **Contextual Embeddings:** Generated using transformer-based models, capturing meaning based on surrounding context.
- **Sentence-Level Embeddings:** Encode entire clauses or sentences into a single vec-

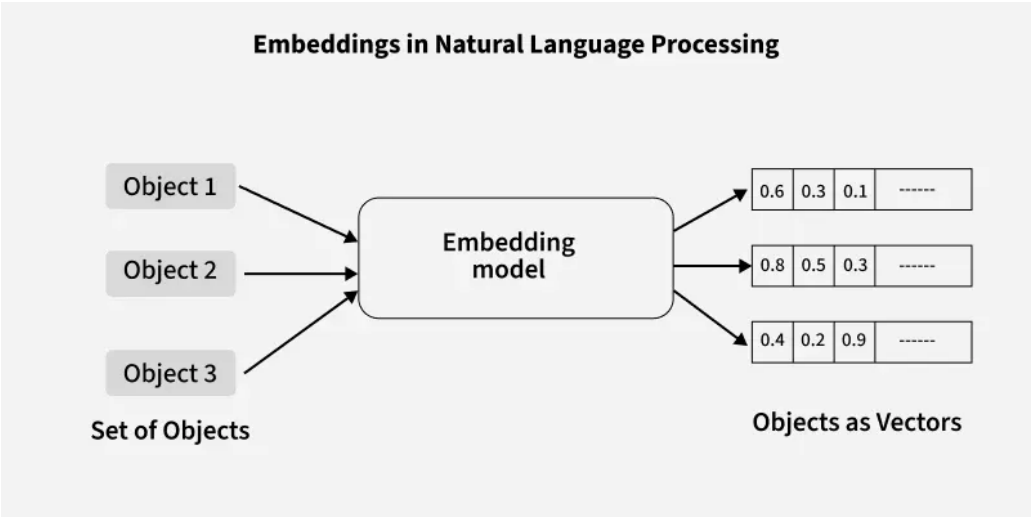

Figure 2.3: Embedding Model

tor representation.

- **Domain-Specific Embeddings:** Fine-tuned on legal corpora to capture specialized terminology and clause relationships.

Similarity Computation

The similarity between a query and a document is computed using cosine similarity:

$$\text{sim}(\mathbf{e}_q, \mathbf{e}_i) = \frac{\mathbf{e}_q \cdot \mathbf{e}_i}{\|\mathbf{e}_q\| \, \|\mathbf{e}_i\|} \tag{2.14}$$

This metric measures the angular distance between vectors, making it robust to differences in magnitude.

2.3.5 Step 4: Dense Retrieval

Dense retrieval leverages embedding similarity to identify relevant documents. For each query, the system computes similarity scores against all document embeddings:

$$\text{score}_i^{\text{dense}} = \cos(\mathbf{e}_q, \mathbf{e}_i) \tag{2.15}$$

The top-k documents are selected:

$$D_{\text{top-}k} = \{d_{i_1}, d_{i_2}, \ldots, d_{i_k}\} \tag{2.16}$$

Dense retrieval is particularly effective in handling:

- Synonyms (e.g., "theft" vs. "stealing")
- Paraphrased queries
- Implicit semantic relationships

However, it may sometimes overlook exact keyword matches, motivating the need for hybrid approaches.

2.3.6 Step 5: Reranking

The initial retrieved set may contain documents that are semantically similar but not optimally ranked. A reranking model refines this ordering by jointly considering query-document interactions.

$$s_i^{\text{rerank}} = f_{\text{rerank}}(q, d_i) \tag{2.17}$$

The final score combines dense and lexical signals:

$$s_i^{\text{final}} = \lambda\, s_i^{\text{dense}} + (1-\lambda)\, s_i^{\text{keyword}} \tag{2.18}$$

where:

- s_i^{dense} represents semantic similarity
- s_i^{keyword} represents lexical relevance (e.g., BM25)
- λ controls the balance between the two

Reranking significantly improves:

- Precision at top-k
- Ordering quality
- Context relevance

2.3.7 Step 6: Hybrid Retrieval

To address the limitations of individual retrieval methods, a hybrid retrieval strategy is employed. This combines:

- **Dense Retrieval:** Captures semantic similarity
- **Sparse Retrieval:** Captures exact keyword matches using methods like BM25 or TF-IDF
- **Reranking:** Refines and reorders candidate documents

Hybrid retrieval ensures robustness across different query types, including:

- Short keyword-based queries

- Long natural language questions
- Ambiguous or partially specified queries

2.3.8 Step 7: Context Integration and Response Generation

The final set of retrieved documents is concatenated with the user query and provided as input to the LLM:

$$\text{Output} = \text{LM}([q; D_{\text{retrieved}}]) \tag{2.19}$$

The LLM generates a response conditioned on both:

- The user query
- The retrieved legal context

This grounding mechanism ensures reduction in hallucinations and improved factual consistency.

2.4 Instruction Tuning and Prompt-Based Learning

Instruction tuning is a technique that aligns language models with human expectations by training them on datasets consisting of (input, output) format. Rather than only learning language patterns, the model learns how to follow explicit instructions, such as ”Summarize Section 55 of the Civil Code” or ”Explain the punishment for theft.”

This is a key component in building interactive, AI-powered assistants. Instruction tuning improves the model’s ability to generalize to new prompts, enhances interpretability, and ensures outputs are relevant and task-specific. In this project, the English version of the

Muluki Samhita is converted into an instruction-tuning dataset, allowing the fine-tuned model to behave like a legal assistant — answering questions, summarizing sections, and assisting users with clarity and structure.

2.5 Model Quantization

Model quantization is a critical step for reducing the memory footprint and improving inference efficiency of large language models. In this study, post-training quantization was applied to the merged FP16 model to enable deployment on resource-constrained hardware.

2.5.1 Quantization Overview

Quantization converts high-precision floating-point weights (typically 16-bit or 32-bit) into lower-bit representations. In this work, 4-bit quantization was employed, significantly compressing the model size while maintaining acceptable performance.

Given a real-valued weight $w \in \mathbb{R}$, quantization maps it to a discrete integer value w_q using:

$$w_q = \text{round}\left(\frac{w - z}{s}\right) \tag{2.20}$$

where:

- s is the scale factor
- z is the zero-point offset

During inference, the quantized weights are approximately reconstructed via dequantization:

$$\hat{w} = s \cdot w_q + z \tag{2.21}$$

2.5.2 4-bit Quantization

In 4-bit quantization, each weight is represented using only 4 bits, allowing for $2^4 = 16$ discrete levels. Compared to FP16 representation, this results in substantial memory savings.

The theoretical compression ratio is:

$$\text{Compression Ratio} = \frac{4}{16} = \frac{1}{4} \tag{2.22}$$

Thus, a 16GB FP16 model can be reduced to approximately 4GB after quantization.

2.5.3 GGUF Quantization Format

The quantized model was stored in GGUF (GPT-Generated Unified Format), which is optimized for efficient inference using `llama.cpp`. GGUF organizes tensor data and metadata to support fast loading, reduced memory usage, and compatibility with CPU-based execution.

2.5.4 Benefits and Trade-offs

Quantization introduces a trade-off between efficiency and accuracy:

- **Advantages:**
  - Reduces model size by approximately 75%
  - Enables deployment on consumer-grade hardware
  - Improves inference speed due to reduced memory bandwidth

- **Limitations:**
  - Minor loss in numerical precision
  - Potential degradation in response quality for complex queries

Despite these trade-offs, 4-bit quantization provides an effective balance between performance and efficiency, making it suitable for real-world deployment of large language models in legal question-answering systems.

CHAPTER 3

LITERATURE REVIEW

3.1 Legal LLM Fine-Tuning Using LoRA and QLoRA in South Asia

Several open-source and academic projects in South Asia have demonstrated the effectiveness of LoRA and QLoRA in adapting general-purpose LLMs for legal reasoning, question answering, and summarization.

- Wakil-G (2025): A Nepali AI-powered legal assistant platform providing instant answers to queries based on Nepalese laws and the Constitution. While it demonstrates potential for legal guidance in Nepal, it is unreliable with frequent downtime and notifications indicating its AI fund/resources are depleted.
- Indian-LawyerGPT (2024): This open-source project fine-tunes Falcon-7B and LLaMA2-7B using QLoRA on a curated dataset of Indian legal instructions. It supports civil, constitutional, and criminal law tasks through a memory-efficient model deployable on limited hardware.
- Finetuning Llama (OpenCode, 2023): This initiative presents a step-by-step fine-tuning process of LLaMA-2-7B using LoRA on Indian law corpora. The project offers Jupyter notebooks and reproducible workflows for legal instruction tuning.
- Legal-Doc-Summarization (2023): A practical implementation where LLaMA-2-7B was fine-tuned using LoRA to summarize Indian court judgments. The model was also used in a retrieval-augmented generation (RAG) pipeline to support document-level question answering.
- Aalap by OpenNyAI (2024): Aalap is an instruction-tuned legal LLM based on

Mistral-7B, trained using LoRA adapters and evaluated on Indian legal exams, including the All India Bar Exam. It demonstrates robust performance in legal comprehension benchmarks.

- INLegalLlama (NyayaAnumana Project): Vats et al. (2024) applied LoRA to LLaMA-2 to build a model trained on thousands of Indian court judgments. The model performs well on verdict prediction and legal explanation tasks, showing LoRA's potential for legal classification and reasoning.
- Legal AI in Pakistan (Nasir et al., 2024): A supervised LoRA fine-tuning framework was applied to LLaMA-3 for legal QA, improving baseline performance by over 20% in understanding Pakistani legal content.

These efforts establish LoRA and QLoRA as accessible, efficient alternatives to full model fine-tuning, particularly suited for legal domains in low-resource settings.

3.2 Research Gap in the Nepali Legal Context

Despite the availability of structured legal documents like the Muluki Dewani Samhita and the Muluki Aparadh Samhita, Nepal lacks any public or academic projects involving LLM fine-tuning — with or without LoRA. There are currently:

- No fine-tuned LLMs on Nepali legal datasets.
- No instruction-tuned models for legal question answering in Nepal.
- No AI-powered tools that utilize QLoRA or LoRA for Nepali civil or criminal law.

Although Wakil-G exists as a Nepali AI legal assistant, its frequent downtime and dependency on limited AI resources highlight its unreliability. This further demonstrates the need for a robust, fine-tuned legal LLM tailored to the Nepali legal system. By applying

QLoRA and instruction tuning on English translations of Nepal's legal codes, this project seeks to build the first AI-powered legal assistant capable of consistently serving lawyers, students, and the general public, while setting a precedent for responsible legal AI in Nepal and other low-resource jurisdictions.

CHAPTER 4

METHODOLOGY

4.1 Overview

This study proposes a multi-stage approach to fine-tune and deploy a Large Language Model (LLM) on Nepalese legal texts that comprises hierarchical data extraction, dataset generation, QLoRA fine-tuning, adapter merging, model quantization, and evaluation.

The proposed system follows a multi-stage pipeline for developing a domain-specific legal language model. The methodology is structured into eight key stages:

1. Data Preparation and Structuring
2. Instruction Dataset Generation
3. Model Fine-Tuning using QLoRA
4. Model Optimization (Adapter Merging and Quantization)
5. Retrieval-Augmented Generation (RAG) Pipeline
6. System Architecture (Frontend and Backend)
7. Evaluation Strategy
8. Deployment

Each stage builds upon the previous one, forming a complete pipeline from raw legal documents to a deployable AI-powered legal assistant.

4.2 Stage 1: Data Preparation and Structuring

The first step involves processing English PDF versions of key legal documents, including:

- National Civil Code, 2017
- National Criminal Code, 2017
- The National Civil Procedure (Code) Act, 2017
- The National Criminal Procedure (Code) Act, 2017
- Constitution of Nepal
- Selected other acts and regulations

4.2.1 Hierarchical Extraction

PDFs were processed using `PyMuPDF`, and text was extracted in a hierarchical structure. Each legal clause or entry was represented as a JSON object:

- `clause_id`: Unique identifier of the clause
- `text`: Full text of the clause
- `legal_document_source`: Source of the legal document
- `part`, `chapter`: Logical segmentation
- `chunk_text`: Combined string containing clause ID, text, part, and source

```
{
  "clause_id": "Clause 1",
  "text": "This is a sample legal clause text.",
  "legal_document_source": "Constitution of Nepal, 2015",
```

```
  "part": "Part 1 (Preliminary)",
  "chapter": null,
  "chunk_text": "Clause 1 | Text: This is a sample legal clause
     ↪ text. | Part: Part 1 (Preliminary) | Legal Source:
     ↪ Constitution of Nepal, 2015"
}
```


4.3 Stage 2: Instruction Dataset Generation

The extracted clauses were processed using an LLM (**Mistral-8B-Instruct-2410**) to generate instruction-output pairs suitable for fine-tuning:


```
{
  "input": "What is the eligibility criteria for X?",
  "output": "Clause Y of the relevant law specifies the
     ↪ eligibility criteria for X."
}
```


The dataset was stored in `.jsonl` format. Instruction-tuning enables the model to follow explicit commands, such as summarizing clauses or answering legal questions.

4.4 Stage 3: Model Fine-Tuning using QLoRA

The base model is **Mistral-7B-Instruct**. Fine-tuning was performed using QLoRA, which loads the base model in 4-bit precision and trains LoRA adapters.

4.4.1 LoRA Configuration

In this study, the LoRA configuration is as follows:

```
lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "k_proj", "v_proj",
                    "o_proj", "gate_proj", "up_proj",
                    "down_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)
```

- **`r=16` (Rank of low-rank matrices):** The rank r determines the size of the trainable matrices $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$ that represent the weight updates. A smaller r reduces the number of trainable parameters and memory usage, while a larger r allows more expressive adaptation to domain-specific data. In this work, $r = 16$ provides a balance between adaptability and efficiency.

- **`lora_alpha=16` (Scaling factor for updates):** LoRA updates are scaled by α/r, controlling the magnitude of the low-rank adaptation. For example, with $\alpha = 32$ and $r = 16$, the scale factor is $32/16 = 2$, meaning the LoRA updates are amplified to have a significant but controlled impact on the frozen base model. This ensures effective adaptation without fully retraining billions of parameters.

- **`target_modules` (Layers receiving LoRA adapters):** These specify which parts of the model are updated. Typically, attention projection matrices (`q_proj, k_proj,`

`v_proj, o_proj`) and MLP layers (`gate_proj, up_proj, down_proj`) are selected, as they capture most of the model's representational power. Updating these modules allows the model to adapt to the legal domain without modifying the entire parameter space.

- **`lora_dropout=0.05` (Dropout applied to LoRA layers):** Dropout is applied to the LoRA adapter outputs during training to prevent overfitting on the domain-specific legal dataset. Here, 5% of the adapter activations are randomly zeroed during each forward pass, encouraging generalization.
- **`bias="none"` (Bias parameters frozen):** No bias terms are trained in this configuration. This reduces the number of trainable parameters and simplifies the adaptation process, focusing learning on the low-rank updates instead of additional biases.
- **`task_type="CAUSAL_LM"` (Task specification):** This setting informs the LoRA implementation that the model is being fine-tuned for causal language modeling, where the model predicts the next token given previous tokens. It ensures that LoRA updates are applied correctly for autoregressive generation tasks, which is the case for question-answering and legal text generation.

Summary: The above configuration enables efficient adaptation of a large language model to the legal domain, controlling memory usage, training time, and expressivity. By updating only targeted low-rank matrices with controlled scaling and dropout, LoRA allows domain-specific fine-tuning without retraining all parameters of an 7B-parameter model.

4.4.2 Training Arguments

The training arguments control how the model is fine-tuned. Each parameter has a specific role in balancing learning stability, memory efficiency, and convergence speed. In this study, the Training Arguments are as follows:

```
training_args = TrainingArguments(
    output_dir="./mistral_7b_qlora",
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
    learning_rate=5e-5,
    eval_strategy="steps",
    eval_steps= 100,
    num_train_epochs=3,
    logging_steps=25,
    max_grad_norm=0.3,
    warmup_ratio=0.05,
    lr_scheduler_type="cosine",
    optim="paged_adamw_8bit",
    fp16=True,
    remove_unused_columns=False,
    report_to="none",
)
```

- **`per_device_train_batch_size=2`**: Number of samples processed per GPU per step. A small batch size is necessary for memory-heavy models like Mistral-7B-Instruct, especially when using 8-bit or 16-bit precision. Although small, the effective batch size can be increased using gradient accumulation.

- **`gradient_accumulation_steps=4`**: Accumulates gradients over multiple steps before performing a weight update. This effectively simulates a batch size of $2 \times 4 = 8$, improving training stability without exceeding GPU memory limits.
- **`learning_rate=5e-5`**: Step size for weight updates. A relatively small learning rate prevents the model from forgetting pretrained knowledge while allowing gradual adaptation to the legal dataset.
- **`num_train_epochs=3`**: Number of passes through the entire dataset. Multiple epochs ensure the model adequately learns domain-specific patterns without overfitting.
- **`max_grad_norm=0.3`**: Gradient clipping threshold. Limits the magnitude of gradients during backpropagation, preventing exploding gradients and stabilizing training.
- **`warmup_ratio=0.05`**: Fraction of total training steps used for linear warmup of the learning rate. Gradually increases the learning rate from 0 to 5×10^{-5} over the first 5% of steps, preventing sudden large updates that may destabilize training early on.
- **`lr_scheduler_type="cosine"`**: Cosine annealing schedule decreases the learning rate following a cosine curve:

  $$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{t\pi}{T}\right)\right)$$

  This allows gradual reduction of the learning rate toward the end of training, improving convergence.
- **`optim="paged_adamw_8bit"`**: Optimizer using 8-bit AdamW with paged mem-

ory. Reduces memory consumption while maintaining optimization quality, making it possible to train large models on limited hardware.

- **`fp16=True`**: Mixed-precision training uses 16-bit floats for activations while keeping critical operations in 32-bit. This accelerates training by 2-3x and reduces GPU memory usage without significant loss of numerical stability.

4.5 Stage 4: Model Optimization (Adapter Merging and Quantization)

4.5.1 LoRA Adapter Merging

For a base weight W_0, LoRA updates BA are merged:

$$W = W_0 + \frac{\alpha}{r} BA = W_0 + 2BA$$

Only targeted modules are updated. Merging results in a FP16 model.

4.5.2 GGUF Conversion and 4-bit Quantization

Merged FP16 weights are converted to GGUF format for efficient inference on CPU/GPU.

The GGUF converted weights of the model are quantized from FP16 to 4-bit:

$$w_q = \text{round}\left(\frac{w - z}{s}\right), \quad \hat{w} = s \cdot w_q + z$$

The resulting GGUF model size reduces approximately by 4x from 16GB to around 4GB, enabling deployment on consumer hardware.

4.6 Stage 5: Retrieval-Augmented Generation (RAG) Pipeline

To enhance factual accuracy and contextual relevance, the chatbot implements a Retrieval-Augmented Generation (RAG) pipeline. This stage ensures that responses are grounded in authoritative legal documents rather than relying solely on the model's pre-trained knowledge. The RAG pipeline is divided into the following steps:

1. **Document Embedding and Storage:** Legal documents are tokenized and embedded using Hugging Face's `sentence-transformers/allMiniLM-L6-v2` model. The resulting dense embeddings are stored in **FAISS** for efficient similarity search.

2. **Query Embedding and Retrieval:** User queries are tokenized and embedded in the same manner as documents. The query embedding is compared against the stored embeddings using cosine similarity:

$$\text{score}_i = \frac{\mathbf{e}_q \cdot \mathbf{e}_i}{\|\mathbf{e}_q\| \, \|\mathbf{e}_i\|}$$

   The top-k most relevant documents are retrieved.

3. **Reranking and Context Selection:** Retrieved documents are reranked using a combination of semantic similarity and keyword relevance to ensure the most pertinent legal clauses are selected.

4. **Context Injection and Response Generation:** The top-ranked documents are concatenated with the user query and provided to the fine-tuned Mistral-7B-Instruct model:

$$\text{LLM Input} = [q; D_{\text{retrieved}}]$$

The model generates a contextually grounded response.

Summary: This RAG pipeline allows the chatbot to retrieve relevant legal documents efficiently and generate accurate, context-aware responses while keeping the methodology concise and practical.

4.7 Stage 6: System Architecture (Frontend and Backend)

4.7.1 System Overview

The system consists of a frontend interface and a backend service. The frontend handles queries and displays responses, while the backend performs RAG retrieval and model inference.

4.7.2 Backend Architecture

1. Receive user query
2. Preprocess and classify
3. Retrieve relevant clauses using RAG
4. Construct augmented prompt
5. Generate response with fine-tuned LLM
6. Return/stream response to frontend

4.7.3 Frontend Architecture

- Accept user queries

- Send requests to backend
- Display responses
- Handle streaming responses

4.7.4 Frontend-Backend Interaction Flow

1. User submits query
2. Query sent to backend via HTTP/WebSocket
3. Backend executes RAG + LLM
4. Response returned/streamed to frontend
5. Frontend renders response

4.7.5 System Flow Diagram

Graphics/FrontendBackendFlow.png

Figure 4.1: Frontend-Backend Interaction in the Web-Based Legal Assistant System

4.7.6 Advantages of Separation

- Modularity for independent development
- Scalability for multiple concurrent users
- Flexibility for streaming responses
- Security by keeping sensitive data on the server

4.8 Stage 7: Evaluation Strategy

The evaluation strategy for the fine-tuned legal language model combines both quantitative metrics and qualitative assessment to ensure the model performs effectively on domain-specific tasks such as legal question-answering.

4.8.1 Quantitative Metrics

1. **Training Loss:** Training loss measures the model's error on the training dataset. For a token sequence $x_1, x_2, \ldots, x_T$, the causal language modeling loss is:

$$\mathscr{L}_{\text{train}} = -\frac{1}{T}\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}) \tag{4.1}$$

   Lower training loss indicates effective learning of patterns from the training data.

2. **Testing Loss:** Testing loss evaluates the model on unseen data, estimating generalization performance:

$$\mathscr{L}_{\text{test}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T_i} \log P_\theta(x_t^{(i)} \mid x_{<t}^{(i)}) \tag{4.2}$$

   A small gap between training and testing loss indicates good generalization.

3. **Perplexity:** Perplexity measures how well the model predicts unseen sequences and is derived from testing loss:

$$\text{Perplexity} = \exp(\mathscr{L}_{\text{test}}) \tag{4.3}$$

Lower perplexity implies more confident and coherent predictions, critical for legal clause generation and reasoning.

4.8.2 Custom Benchmarking

1. **Clause Mapping Accuracy (CMA):** CMA evaluates the model's ability to retrieve or generate the exact legal clause(s) corresponding to a user query. Each benchmark entry consists of a query and its ground truth clause(s) from legal texts.
2. **Contextual Accuracy (CA):** CA measures the model's ability to generate contextually correct answers, even when the exact wording of the clause is not reproduced. It emphasizes understanding and reasoning over the legal text.
3. **LLM-as-Judge Evaluation:** A separate pre-trained LLM automatically scores model outputs based on CMA, CA, completeness, and clarity, providing a scalable evaluation.
4. **Human Evaluation:** Human is in the loop and assess a subset of queries for correctness, consistency, and applicability, ensuring reliability in ambiguous or complex cases.

4.9 Stage 8: Deployment

The quantized 4-bit GGUF model is deployed for inference, providing an interactive web-based legal assistant with efficient performance on consumer hardware.

CHAPTER 5

EVALUATION AND BENCHMARKING

5.1 Model Training Overview

The Nepal-Legal model is a domain-specific fine-tuned version of the Mistral-7B architecture. The model was trained on a curated dataset of Nepali legal texts using QLoRA-based fine-tuning.

5.1.1 Training and Validation Loss

Training and validation loss were tracked throughout the training process to monitor learning behavior and generalization. Training loss reflects how well the model fits the training data, while validation loss evaluates performance on unseen data.

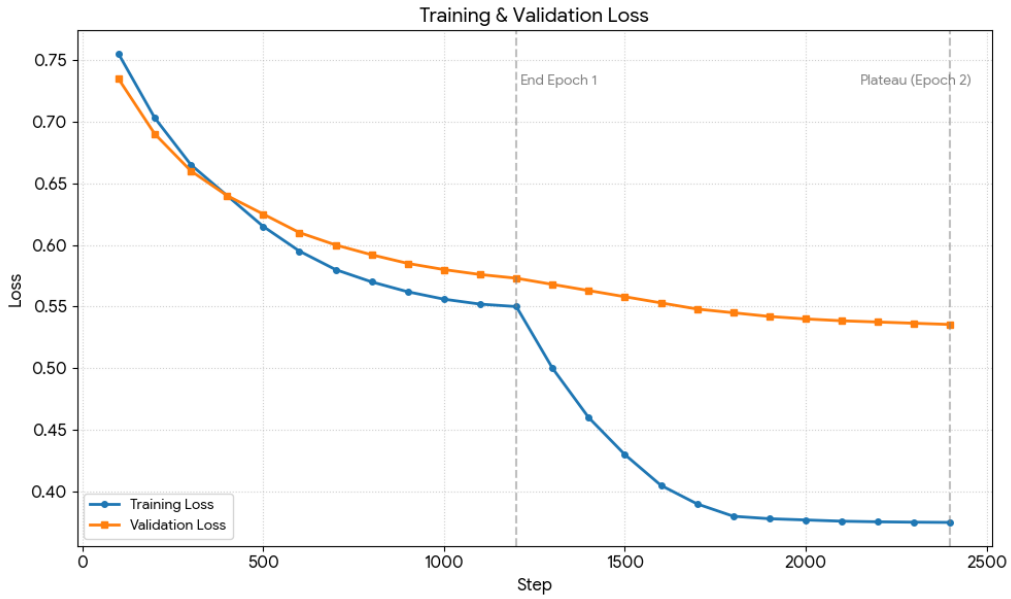

Figure 5.1: Training vs Validation Loss over training steps

Observation: Both training and validation loss show a steady downward trend, indicating effective learning. Training loss decreases from 0.7549 to 0.3750, showing strong learning and convergence. Validation loss reduces from 0.7346 to 0.5355, indicating steady but slower generalization. The final gap (0.16) remains moderate, suggesting minimal overfitting and stable performance.

5.1.2 Perplexity Analysis

Perplexity was used as an additional metric to evaluate how confidently the model predicts text. Lower perplexity indicates better predictive performance and more coherent output generation.

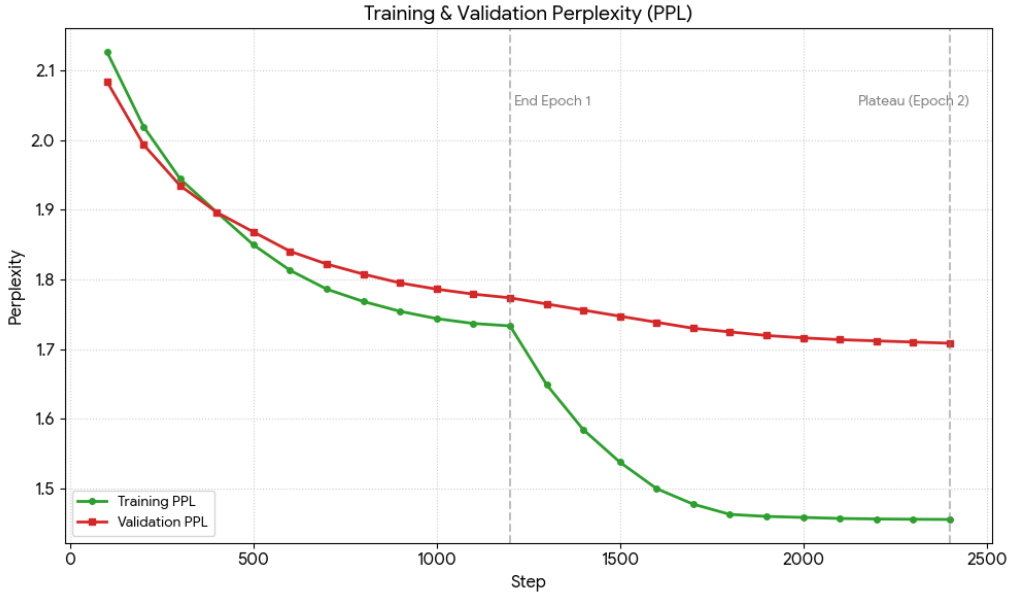

Figure 5.2: Perplexity over training steps

Observation: The perplexity curve decreases consistently during training, aligning with the improvement observed in validation loss. Training perplexity drops from 2.13 to 1.46, reflecting significant improvement in prediction confidence. Validation perplexity decreases from 2.08 to 1.71, showing consistent generalization across epochs. The small difference (0.25) between them confirms the model generalizes well without overfitting.

5.2 Limitations of Standard Metrics

Traditional NLP metrics such as **ROUGE** and **BERTScore** often fail in the legal domain. These metrics prioritize linguistic similarity over legal precision. In law, a single word change or a slightly different section citation can render an answer legally incorrect, even if the ROUGE score remains high. Consequently, a more rigorous, domain-specific framework is required.

5.3 Custom Benchmarking Parameters

To ensure legal reliability, we propose two primary parameters and a calculated composite score for evaluation:

- **CMA (Citation/Clause Mapping Accuracy):** Measures the model's ability to map a legal query to the exact correct section, clause, or article of the law.
- **CA (Content Accuracy):** Measures the substantive correctness of the legal advice or summary provided, ensuring no "hallucinations" regarding legal rights or obligations.
- **WMS (Weighted Model Score):** It is the calculated composite of both CMA and CA.

$$\text{WMS} = 0.6 \times \text{CMA} + 0.4 \times \text{CA} \tag{5.1}$$

5.4 Evaluation Framework and Scoring Metrics

We utilize the **Weighted Model Score (WMS)** to categorize answer quality.

Tier	WMS Range	Interpretation
Perfect	≥ 0.95	Correct law, section, and content. Production-ready.
Excellent	0.80 – 0.94	Minor citation gap but substantively correct.
Good	0.65 – 0.79	Mostly correct with small errors.
Partial	0.45 – 0.64	Material gaps; needs human review.
Weak	0.20 – 0.44	Significant errors in law or content.
Failure	< 0.20	Wrong law, wrong answer, or inappropriate refusal.

Figure 5.3: Evaluation Framework

5.5 Models Under Test

The benchmarking compares the domain-specific fine-tuned model against the general-purpose base model.

Attribute	Nepal-Legal (Fine tuned)	Mistral-7B-BASE v0.1 (Local)
Model Type	Domain-specific fine-tuned LLM	General-purpose base LLM
Architecture	Mistral-7B fine-tuned on Nepali law	Mistral-7B-v0.1 untuned
Quantization	Q4_K_M (GGUF)	Q4_K_M (GGUF)
Serving	Cloudflare Tunnel / External API	llama-cpp-python (local)
Context Window	Custom legal context	2,048 tokens (capped)
RAG	Tested without and with	Tested without and with
Avg. Latency	0.88s / 11.12s	~50s per answer

Figure 5.4: Models Under Testing

5.6 Benchmarking Run 1: Without RAG

Initial tests were conducted using the models' internal parametric memory only.

5.6.1 Overall Scores (No RAG)

Metric	Nepal-Legal	Verdict	Mistral-7B	Verdict
μCMA (Citation Accuracy)	78.2%	Good	38.6%	Weak
μCA (Content Accuracy)	71.1%	Good	61.8%	Partial
μWMS Overall	75.4%	Good	47.9%	Weak
Citation Rate	76.0%	—	0.0%	—
Refusal Accuracy	100.0%	Perfect	0.0%	Failure
Perfect Answers (WMS≥0.95)	24 / 55	—	5 / 55	—
Excellent Answers	22 / 55	—	14 / 55	—
Failure Answers (WMS<0.20)	2 / 55	—	14 / 55	—
WMS Delta (Nepal − Mistral)	+27.5%	Nepal Wins	—	—

Figure 5.5: No RAG Benchmarking

5.6.2 Per Category Breakdown (No RAG)

Category	N	Nepal CMA	Nepal CA	Nepal WMS	Mistral CMA	Mistral CA	Mistral WMS	Winner
Constitutional Law	5	72.0%	70.0%	71.0%	92.0%	96.0%	94.0%	Mistral ✓
Civil Procedure	5	68.0%	68.0%	68.0%	28.0%	48.0%	36.0%	Nepal ✓
Criminal Procedure	5	84.0%	56.0%	73.0%	36.0%	60.0%	46.0%	Nepal ✓
Criminal Law	5	92.0%	86.0%	90.0%	54.0%	76.0%	63.0%	Nepal ✓
Family Law	5	60.0%	62.0%	61.0%	56.0%	62.0%	58.0%	Nepal ✓
Contract Law	5	92.0%	78.0%	86.0%	44.0%	82.0%	59.0%	Nepal ✓
Defamation Law	5	76.0%	72.0%	74.0%	40.0%	64.0%	50.0%	Nepal ✓
Tort Law	5	92.0%	84.0%	89.0%	22.0%	74.0%	43.0%	Nepal ✓
Jurisdiction & Courts	5	56.0%	38.0%	49.0%	20.0%	62.0%	37.0%	Nepal ✓
Rights & Equality	5	68.0%	68.0%	68.0%	32.0%	56.0%	42.0%	Nepal ✓
Should Refuse	5	100.0%	100.0%	100.0%	0.0%	0.0%	0.0%	Nepal ✓

Figure 5.6: No RAG Benchmarking with categories

5.6.3 Radar Chart for No RAG

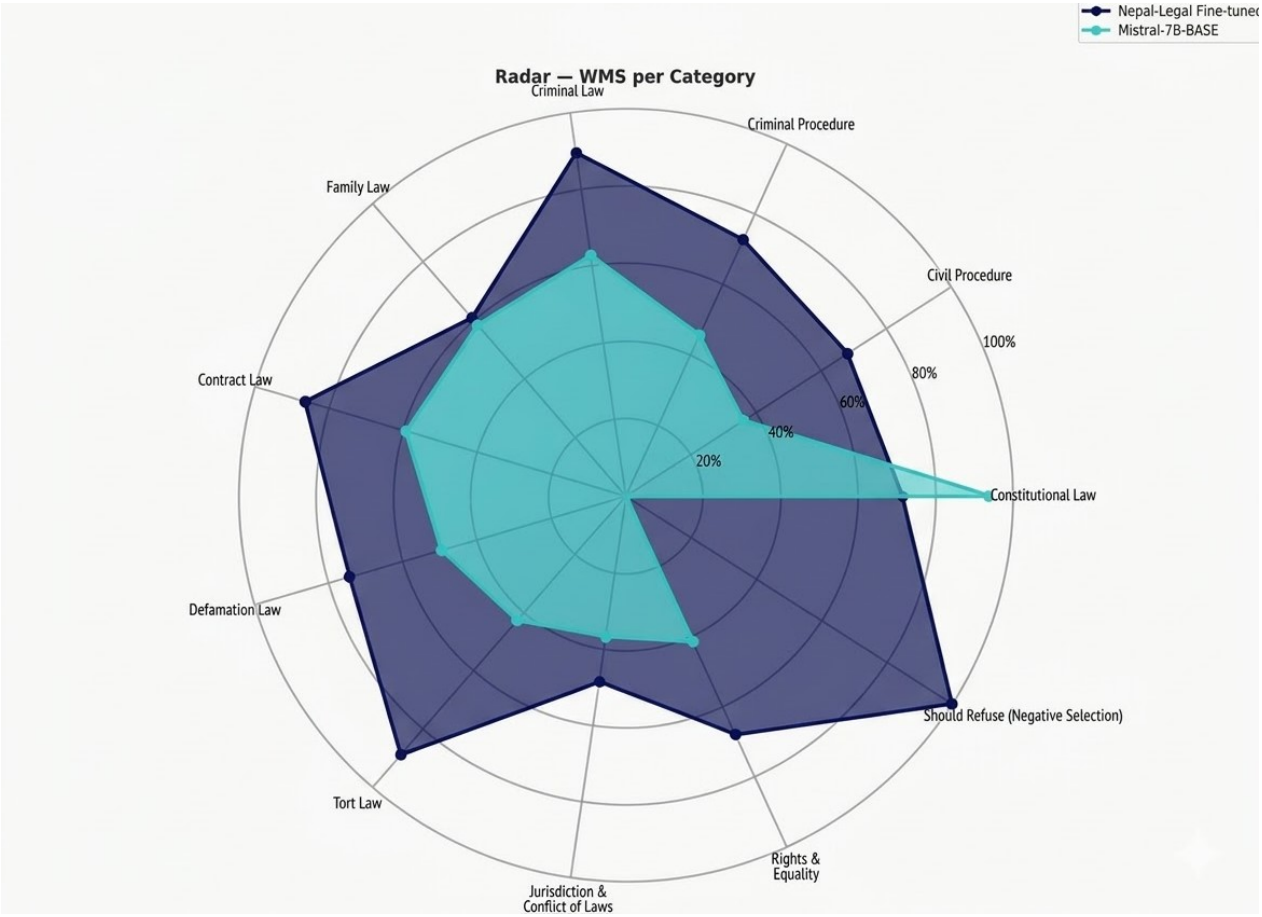

Figure 5.7: Radar Chart (No RAG)

5.6.4 Notable Observations (Without RAG)

The following observations summarize the performance characteristics of the Nepal-Legal model compared to Mistral 7B base model in the absence of Retrieval-Augmented Generation (RAG):

Strong Domain Performance: Nepal-Legal model achieved perfect scores in Criminal Law, Contract Law, Tort Law, and Defamation Law categories. This indicates highly effective domain-specific fine-tuning and strong alignment with structured legal datasets.

Isolated Mistral Advantage: Mistral outperformed Nepal-Legal model only in Constitutional Law (WMS = 0.94). This is likely due to its broader general knowledge of constitutional principles, whereas Nepal-Legal showed inaccuracies in clause citation for this category.

Refusal Handling Capability: Nepal-Legal model achieved a 100% success rate (5/5) on refusal-based queries, while Mistral showed 0% refusal accuracy. This highlights the importance of fine-tuning for safe and controlled responses in production systems.

Weakness in Multi-Law Reasoning: The Jurisdiction & Courts category was the weakest for Nepal-Legal model (WMS = 0.49). This suggests limitations in handling queries that require cross-referencing across multiple legal provisions or documents.

Mistral Performance Characteristics: Mistral achieved a higher Contextual Accuracy (CA = 61.8%) compared to its Clause Mapping Accuracy (CMA = 38.6%). This indicates that while it can generate contextually relevant answers, it lacks precise citation capability for Nepali legal statutes.

5.7 Benchmarking Run 2: With RAG

Retrieval-Augmented Generation (RAG) was implemented to provide the models with relevant legal context before generation.

5.7.1 Overall Scores (With RAG)

Metric	Nepal-Legal	Verdict	Mistral-7B	Verdict
μCMA (Citation Accuracy)	90.6%	Excellent	51.8%	Partial
μCA (Content Accuracy)	91.3%	Excellent	57.8%	Partial
μWMS Overall	90.8%	Excellent	54.2%	Partial
Citation Rate	74.0%	—	0.0%	—
Refusal Accuracy	100.0%	Perfect	0.0%	Failure
Perfect Answers (WMS≥0.95)	41 / 55	—	0 / 55	—
Excellent Answers	7 / 55	—	19 / 55	—
Failure Answers (WMS<0.20)	2 / 55	—	12 / 55	—
WMS Delta (Nepal − Mistral)	+36.6%	Nepal Wins	—	—

Figure 5.8: Benchmarking Score with RAG

5.7.2 Per Category Breakdown (With RAG)

Category	N	Nepal CMA	Nepal CA	Nepal WMS	Mistral CMA	Mistral CA	Mistral WMS	Winner
Constitutional Law	5	**86.0%**	90.0%	**88.0%**	**76.0%**	74.0%	**75.0%**	**Nepal ✓**
Civil Procedure	5	**70.0%**	70.0%	**70.0%**	**53.0%**	58.0%	**55.0%**	**Nepal ✓**
Criminal Procedure	5	**100.0%**	100.0%	**100.0%**	**76.0%**	78.0%	**77.0%**	**Nepal ✓**
Criminal Law	5	**98.0%**	100.0%	**99.0%**	**55.0%**	65.0%	**59.0%**	**Nepal ✓**
Family Law	5	**66.0%**	68.0%	**67.0%**	**52.0%**	58.0%	**54.0%**	**Nepal ✓**
Contract Law	5	**90.0%**	92.0%	**91.0%**	**59.0%**	72.0%	**64.0%**	**Nepal ✓**
Defamation Law	5	**96.0%**	96.0%	**96.0%**	**56.0%**	62.0%	**58.0%**	**Nepal ✓**
Tort Law	5	**100.0%**	96.0%	**98.0%**	**76.0%**	80.0%	**78.0%**	**Nepal ✓**
Jurisdiction & Courts	5	**90.0%**	92.0%	**91.0%**	**35.0%**	47.0%	**40.0%**	**Nepal ✓**
Rights & Equality	5	**100.0%**	100.0%	**100.0%**	**32.0%**	42.0%	**36.0%**	**Nepal ✓**
Should Refuse	5	**100.0%**	100.0%	**100.0%**	**0.0%**	0.0%	**0.0%**	**Nepal ✓**

Figure 5.9: Benchmarking with categories using RAG

5.7.3 Radar Chart for RAG based implementation

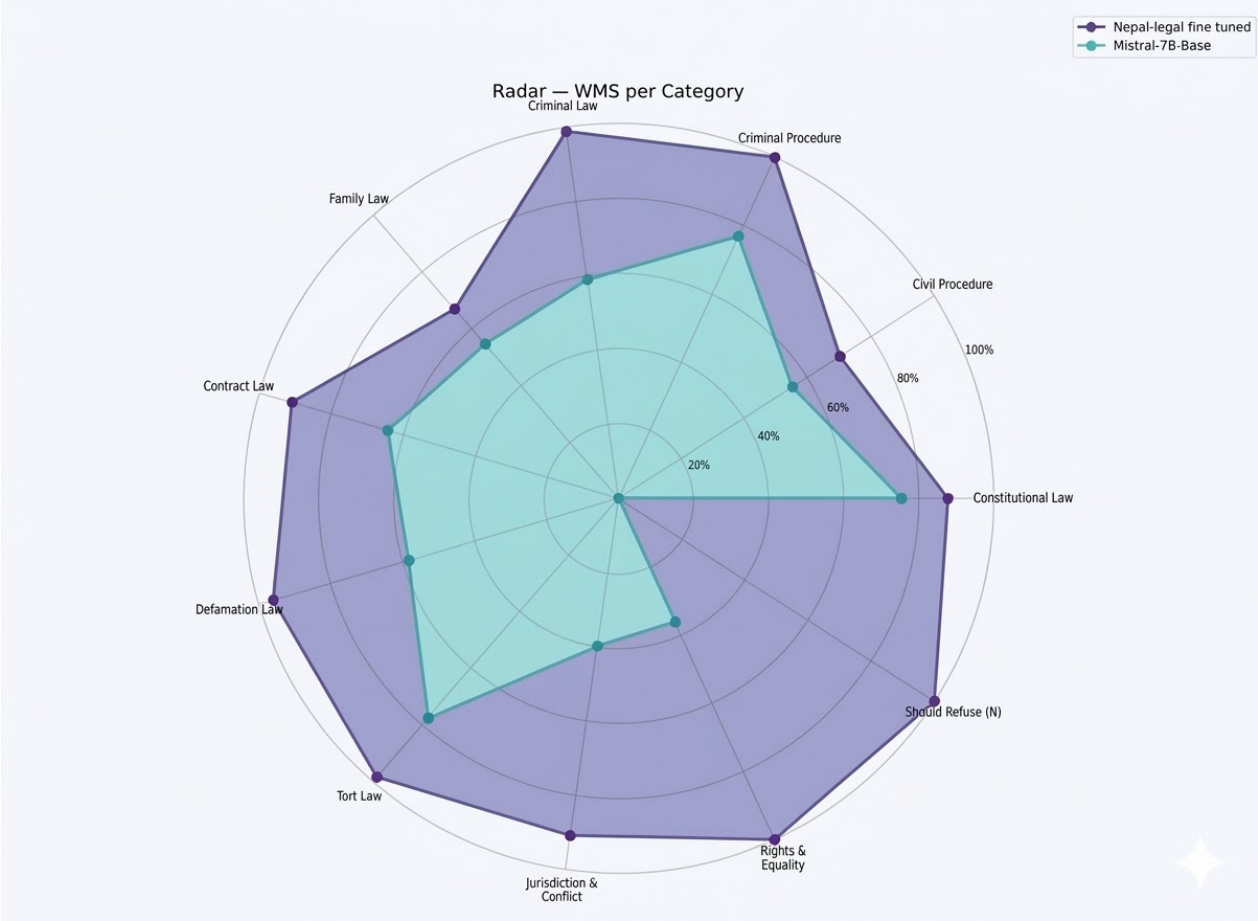

Figure 5.10: Radar Chart (With RAG)

5.7.4 Noteable Observation (With RAG)

The following observations summarize the performance characteristics of the Nepal-Legal model compared to Mistral 7B base model in the presence of Retrieval-Augmented Generation (RAG):

Significant Performance Gains: Nepal-Legal achieves a perfect WMS of 1.00 in Criminal Procedure, Criminal Law, Rights & Equality, and Should Refuse categories. This represents a substantial improvement over the no-RAG baseline and demonstrates the effectiveness of retrieval in grounding responses.

Performance Degradation in General Model: Mistral's performance in Constitutional Law decreases from WMS = 0.94 (without RAG) to WMS = 0.75 (with RAG). This suggests that the retrieval pipeline may introduce conflicting or less relevant context when used with a general-purpose model.

Failure Cases Analysis: The remaining failure cases for Nepal-Legal (Res Judicata and Parental Authority) appear to be scope-related. In these instances, the model correctly identifies that the queries may fall outside the specific domain of Nepali law.

Transformation in Answer Quality: The number of Perfect-tier responses for Nepal-Legal increases from 24 to 41 after applying RAG. This significant improvement indicates that RAG elevates the model from a "Good" level to an "Excellent-to-Perfect" level for domain-specific legal queries.

5.8 RAG Impact Assessment

The integration of RAG significantly boosted the performance of the fine-tuned model across all legal categories.

5.8.1 Score Improvements from RAG

Model / Metric	No-RAG CMA	RAG CMA	ΔCMA	No-RAG CA	RAG CA	ΔCA	ΔWMS
Nepal-Legal	78.2%	90.6%	+12.4%	71.1%	91.3%	+20.2%	+20.5%
Mistral-7B-BASE	38.6%	51.8%	+13.2%	61.8%	57.8%	−4.0%	+6.4%

Figure 5.11: Score Improvement with RAG

5.8.2 Per Category Score Improvements from RAG

Category	WMS No-RAG	WMS RAG	Δ WMS	Assessment
Constitutional Law	71.0%	88.0%	+24.0%	Strong gain
Civil Procedure	68.0%	70.0%	+2.9%	Marginal gain
Criminal Procedure	73.0%	100.0%	+37.0%	Maximum gain
Criminal Law	90.0%	99.0%	+10.0%	Near-perfect
Family Law	61.0%	67.0%	+9.8%	Moderate gain
Contract Law	86.0%	91.0%	+5.8%	Solid gain
Defamation Law	74.0%	96.0%	+29.7%	Strong gain
Tort Law	89.0%	98.0%	+10.1%	Near-perfect
Jurisdiction & Courts	49.0%	91.0%	+85.7%	Largest gain
Rights & Equality	68.0%	100.0%	+47.1%	Maximum gain
Should Refuse	100.0%	100.0%	0.0%	Already perfect

Figure 5.12: Per Category Score Improvement with RAG

5.8.3 Notable Observations from RAG Score Improvements

The following observations summarize the impact of RAG on overall model performance:

Overall Performance Boost: RAG significantly improves performance for both models, with Nepal-Legal achieving a +20.5% increase in WMS. This confirms the strong effectiveness of retrieval in domain-specific tasks.

Superior Gains in Domain-Specific Model: While both models benefit from RAG, Nepal-Legal shows more meaningful improvements in Contextual Accuracy (+20.2%), indicating better utilization of retrieved legal context compared to Mistral.

Mistral Limitation in Context Usage: Mistral shows a decline in Contextual Accuracy (-4.0%), despite improvements in Clause Mapping Accuracy. This suggests that while retrieval helps it identify relevant clauses, it struggles to effectively integrate the retrieved context into coherent responses.

5.9 Final Verdict

The evaluation and benchmarking results clearly demonstrate the effectiveness of domain-specific fine-tuning combined with Retrieval-Augmented Generation (RAG) for legal question-answering systems.

Firstly, the Nepal-Legal model consistently outperforms the general-purpose Mistral-7B base model across both benchmarking runs. Without RAG, the fine-tuned model already exhibits strong domain alignment, achieving perfect scores in multiple legal categories and significantly outperforming the base model in Clause Mapping Accuracy (CMA). This highlights the importance of specialized training on structured legal datasets.

Secondly, the integration of RAG leads to a substantial improvement in overall perfor-

mance. The Nepal-Legal model achieves a +20.5% increase in Weighted Model Score (WMS), along with notable gains in Contextual Accuracy (CA). These improvements confirm that RAG effectively enhances factual grounding, enabling the model to retrieve and utilize relevant legal context during response generation.

In contrast, the general-purpose Mistral model shows limited benefit from RAG, with a decline in contextual accuracy. This indicates that retrieval alone is insufficient without domain-specific alignment, and may even introduce noise when the model lacks the capability to properly interpret retrieved context.

Furthermore, category-wise analysis reveals that RAG is particularly impactful in complex legal domains such as Jurisdiction & Courts and Rights & Equality, where multi-clause reasoning and cross-referencing are required. The increase in Perfect-tier responses from 24 to 41 further demonstrates that RAG transforms the system from a moderately accurate model into a highly reliable legal assistant.

Finally, the combined evaluation framework using CMA, CA, and WMS proves effective in capturing both legal correctness and contextual reasoning, addressing the limitations of traditional NLP metrics.

Conclusion: The study establishes that the optimal approach for building reliable legal AI systems is the integration of domain-specific fine-tuning with retrieval-based augmentation. The Nepal-Legal model with RAG achieves high accuracy, strong legal grounding, and robust real-world applicability, making it suitable for deployment as an AI-powered legal assistant.

REFERENCES

[1] Wakil-G. (2026) Wakil-g: Ai legal assistant for nepalese law. [Online]. Available: https://wakilg.com/

[2] Indian-LawyerGPT Team, "Indian-lawyergpt: A fine-tuned falcon and llama2 for indian legal tasks," https://github.com/indianlawyergpt, 2024, open-source legal LLM using QLoRA on Indian laws.

[3] OpenCode Contributors, "Finetuning llama-2-7b on indian legal corpus," https://github.com/opencode/llama-india-law, 2023, includes LoRA-based Jupyter workflows.

[4] Unknown, "Legal-doc-summarization using llama-2-7b," https://github.com/legaldocsummary, 2023, summarizing Indian court judgments using LoRA and RAG.

[5] OpenNyAI, "Aalap: Instruction-tuned mistral-7b for indian legal exams," https://github.com/opennyai/aalap, 2024, evaluated on All India Bar Exam questions.

[6] R. Vats *et al.*, "Inlegalllama: Lora-tuned legal llm for indian judgments," https://github.com/nyayaanumana, 2024, trained on thousands of Indian court cases.

[7] A. Nasir and M. Khan, "Legal ai in pakistan: Llama-3 fine-tuning with lora," https://github.com/pakistan-legal-ai, 2024, achieved 20% improvement in legal QA tasks.

[1–7]